

Project Overview

1. Purpose and Motivation

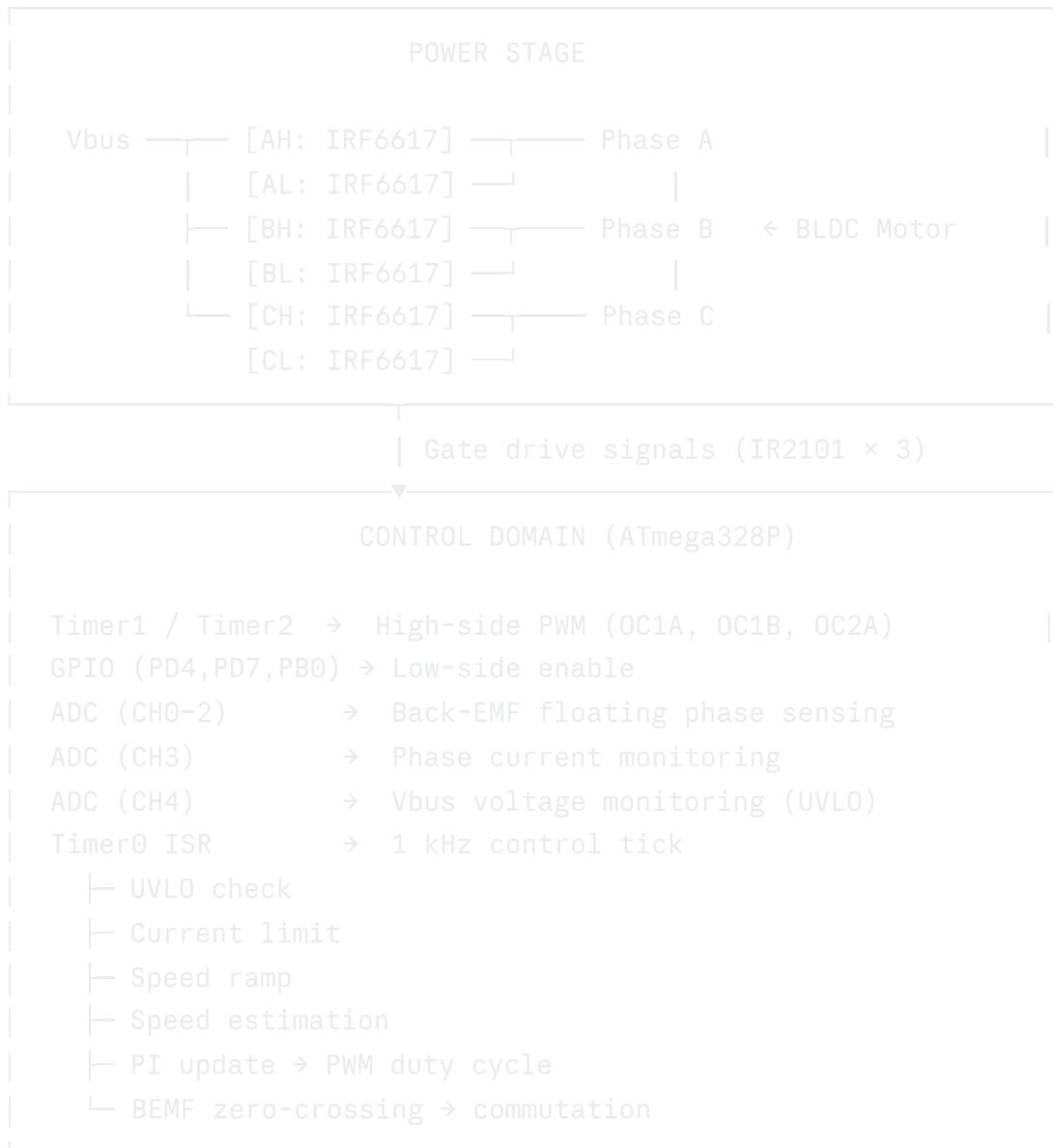
This project implements a complete **sensorless Electronic Speed Controller (ESC)** for a three-phase Brushless DC (BLDC) motor, built entirely on an ATmega328P microcontroller (Arduino UNO R3) using bare-metal AVR C. The goal was to understand and implement every layer of a real motor controller — from register-level timer configuration and interrupt-driven control loops, to back-EMF zero-crossing detection, six-step commutation sequencing, and closed-loop PI speed regulation — without relying on any motor-control library or high-level abstraction.

BLDC motors are preferred in applications such as drones, electric vehicles, robotic actuators, and industrial drives because of their high efficiency, high power density, and low maintenance profile (no brushes to wear). Driving them correctly, however, requires precise rotor-position knowledge and fast switching logic. High-performance applications use dedicated hardware sensors (Hall-effect, encoders, resolvers); sensorless controllers infer rotor position from the motor's own back-electromotive force (back-EMF), reducing cost and mechanical complexity at the expense of firmware sophistication.

This project demonstrates the full sensorless drive chain on constrained 8-bit hardware — a deliberate choice that forces careful management of ISR timing, ADC scheduling, and CPU budget.

2. System Architecture Summary

The system consists of two interacting domains: the **power stage**, which switches motor phase currents at voltages up to the Vbus rail, and the **control domain**, running on the ATmega328P at 5 V logic.



3. Design Decisions

3.1 Bare-Metal AVR C

The firmware is written against AVR hardware registers directly (`TCCR1A`, `ADMUX`, `OCR1A`, etc.) rather than the Arduino HAL. This was intentional: understanding the exact timer prescaler values, ADC conversion timing, interrupt vector configuration, and GPIO register mappings is essential for a functioning motor controller. The Arduino HAL adds indirection and timing uncertainty that is acceptable for general I/O but problematic for motor control.

3.2 Software Back-EMF Detection vs. Hardware Comparator

The schematic includes an **LM393 dual comparator**, which can be used for hardware zero-crossing detection. In the current firmware, zero-crossing detection is performed in software by

the ADC: the floating phase voltage is sampled and compared against a 512-count virtual neutral (midscale of a 10-bit ADC with 5 V reference). This avoids needing a comparator interrupt but consumes ADC conversion time within the ISR. The LM393 path is present for a future upgrade to comparator-interrupt-driven ZC detection, which would improve timing accuracy at higher speeds.

3.3 Single-ISR Architecture

All time-critical operations — UVLO check, current limit, speed ramp, speed estimation, PI update, BEMF polling, and commutation triggering — are consolidated into a **single Timer0 compare-match ISR**. This eliminates inter-ISR priority conflicts and makes execution order deterministic. The trade-off is that the ISR must complete within one timer period ($\sim 100\ \mu\text{s}$ at the configured prescaler and compare value).

3.4 PI Controller Without Derivative Term

A full PID controller was considered but the derivative term was omitted for two reasons. First, speed estimation from zero-crossing timing is inherently noisy on 8-bit hardware, and the derivative of a noisy signal produces excessive duty-cycle jitter. Second, BLDC speed response is dominated by inertia and back-EMF, making the PI response sufficient for smooth closed-loop operation. The derivative term can be added once speed estimation is filtered.

3.5 Open-Loop Startup Ramp

Sensorless back-EMF detection is unreliable at standstill and very low speeds because back-EMF magnitude is proportional to rotor velocity ($V_{\text{BEMF}} = K_e \times \omega$). The firmware therefore starts the motor with an open-loop forced-commutation sequence at a long initial period (`START_PERIOD = 400` ticks), then linearly shortens the period (`RAMP_RATE = 1` tick per `RAMP_INTERVAL = 20` ISR ticks) until the target period is reached. At that point, back-EMF amplitude is sufficient for reliable zero-crossing detection and the loop transitions to closed-loop operation.

4. Scope and Limitations

This implementation is an educational and portfolio-grade proof-of-concept. The following limitations apply:

- The ATmega328P's 10-bit ADC and 8-bit timer resolution constrain speed estimation accuracy and PWM resolution.
- The software zero-crossing detection is susceptible to ADC noise; hardware comparator detection (via the LM393) is more robust.
- The PI controller's floating-point arithmetic is computationally expensive on an 8-bit MCU; a fixed-point implementation would be preferable in production.

- No fault recovery logic is implemented; once a fault latch is set (e.g., UVLO), the motor remains disabled until a hardware reset.
 - The current sensing circuit measures a proxy signal; a proper shunt + op-amp circuit with calibration would be required for accurate current measurement.
-

5. Intended Audience

This documentation is written for:

- Embedded systems engineers and students studying motor control
- Makers building their own ESC hardware
- Engineers evaluating the project for further development (STM32 migration, FOC upgrade)
- Anyone interested in understanding what happens inside a commercial ESC at the register level